INTEGRATED E-COMMERCE ORDER PROCESSING AND RECONCILIATION

Python Programming Project Report

Student Name	Kamatala Kowmya Sri
Roll Number	26000169
Branch / Section	CSE / 3
Institution	NRI University
Academic Year	2026–2027

Prepared as an academic case study with sample data, runnable Python code, test cases, and illustrative console-output screenshots.

1. Problem Statement

E-commerce businesses must process customer orders accurately while applying discounts, dispatching products, recording returns, updating stock, and reconciling the expected order totals against the amounts recorded as paid. Manual processing can cause stock errors, incorrect refunds, and revenue discrepancies. This project develops a Python program that processes a small set of sample orders, calculates discounts and net revenue, tracks dispatches and returns, updates inventory, identifies payment discrepancies, and prints a final inventory summary.

2. Objectives

• Maintain product prices and opening stock in a dictionary.

• Calculate order subtotals, discount amounts, and final payable totals.

• Record dispatch quantities and reduce available stock only for dispatched items.

• Process returns, calculate refunds using the discounted unit price, and add returned items back to stock.

• Compare expected net payments with recorded payments and flag discrepancies.

• Calculate net revenue after refunds and present a final stock and reconciliation report.

3. Assumptions

• All values are sample data; currency is represented in Indian rupees (■).

• Each order contains one product type and a positive quantity.

• A discount percentage applies to the full order subtotal before returns.

• Only dispatched units reduce stock; accepted returns are added back to stock.

• Refunds use the discounted unit price and cannot exceed the value of dispatched units.

• Recorded payment is the amount received for the original order; refunds are processed separately.

• The program flags any difference greater than ■0.01 between expected and recorded payment.

• The program uses in-memory sample data and does not connect to a live store, payment gateway, or courier service.

4. Algorithm

1. Start the program and create the product inventory and sample order records.

2. Set total gross sales, discounts, refunds, net revenue, and discrepancy count to zero.

3. For each order, validate the product and quantity against available stock.

4. Calculate subtotal = unit price × ordered quantity; calculate discount and expected payable amount.

5. Compare expected payable amount with the recorded payment and report any difference.

6. Dispatch the requested quantity if stock is sufficient; reduce inventory by dispatched quantity.

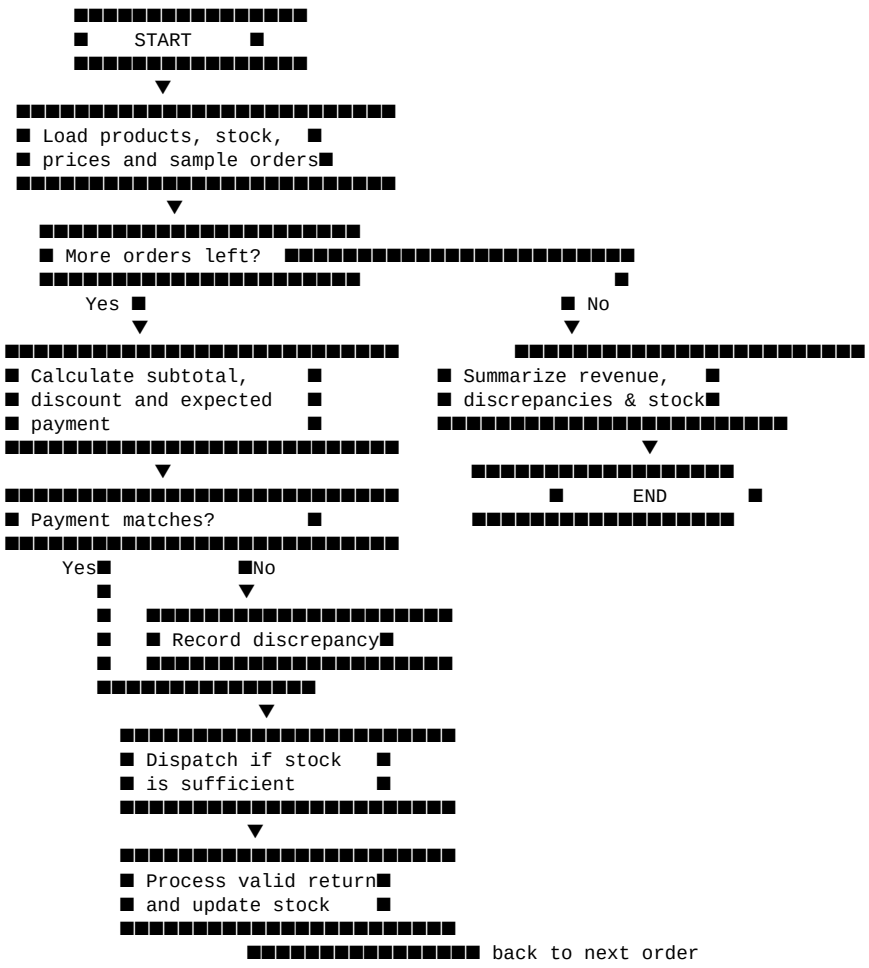
7. Validate the return quantity, calculate refund at the discounted unit price, and add returned units back to inventory.

8. Add the order's discounted sales less refunds to net revenue.

9. After all orders, display order-level results, discrepancies, revenue totals, and final inventory.

10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    inventory ← product names with price and opening stock
    orders ← sample order records
    net_revenue ← 0
    total_discounts ← 0
    total_refunds ← 0

    FOR each order IN orders
        subtotal ← price × quantity
        discount ← subtotal × discount_percent / 100
        expected_payment ← subtotal - discount

        IF absolute(expected_payment - recorded_payment) > 0.01
            report payment discrepancy
        END IF

        IF quantity <= available_stock
            dispatch quantity
            available_stock ← available_stock - quantity
        ELSE
            report insufficient stock and skip dispatch
        END IF

        IF return_quantity is valid and <= dispatched quantity
            refund ← return_quantity × discounted unit price
            available_stock ← available_stock + return_quantity
        ELSE
            refund ← 0
            report invalid return
        END IF

        net_revenue ← net_revenue + expected_payment - refund
        accumulate discounts and refunds
    END FOR
```

```
    display order results
    display total revenue and reconciliation discrepancies
    display final inventory
END
```

7. Python Source Code

```
from collections import defaultdict

# Sample inventory: product -> price and opening stock
inventory = {
    "Keyboard": {"price": 800.00, "stock": 20},
    "Mouse": {"price": 500.00, "stock": 15},
    "Headphones": {"price": 1500.00, "stock": 10},
}

# Each order has an expected payment, a dispatch quantity, and a return quantity.
orders = [
    {"id": "ORD101", "product": "Keyboard", "qty": 2, "discount": 10,
     "paid": 1440.00, "dispatch": 2, "return": 1},
    {"id": "ORD102", "product": "Mouse", "qty": 3, "discount": 5,
     "paid": 1425.00, "dispatch": 3, "return": 0},
    {"id": "ORD103", "product": "Headphones", "qty": 1, "discount": 0,
     "paid": 1400.00, "dispatch": 1, "return": 0},  # discrepancy: expected 1500
    {"id": "ORD104", "product": "Keyboard", "qty": 1, "discount": 0,
     "paid": 800.00, "dispatch": 1, "return": 0},
]

total_gross = 0.0
total_discounts = 0.0
total_refunds = 0.0
net_revenue = 0.0
discrepancies = []
order_results = []

for order in orders:
    product = order["product"]
    if product not in inventory:
        order_results.append((order["id"], "ERROR: unknown product"))
        continue

    price = inventory[product]["price"]
    qty = order["qty"]
    dispatch_qty = order["dispatch"]
    return_qty = order["return"]

    if qty <= 0 or dispatch_qty < 0 or dispatch_qty > qty:
        order_results.append((order["id"], "ERROR: invalid quantity/dispatch"))
        continue

    subtotal = price * qty
    discount = subtotal * order["discount"] / 100
    expected = round(subtotal - discount, 2)
    difference = round(order["paid"] - expected, 2)

    total_gross += subtotal
    total_discounts += discount

    if abs(difference) > 0.01:
        discrepancies.append((order["id"], expected, order["paid"], difference))

    # Only dispatched goods reduce stock.
    if dispatch_qty > inventory[product]["stock"]:
        order_results.append((order["id"], "ERROR: insufficient stock"))
        continue
    inventory[product]["stock"] -= dispatch_qty

    # Refunds are based on the discounted price per unit.
    if return_qty < 0 or return_qty > dispatch_qty:
        order_results.append((order["id"], "ERROR: invalid return quantity"))
        # Roll back dispatch when the return data is invalid.
        inventory[product]["stock"] += dispatch_qty
        continue

    discounted_unit_price = price * (1 - order["discount"] / 100)
    refund = round(discounted_unit_price * return_qty, 2)
    inventory[product]["stock"] += return_qty
    total_refunds += refund
    net_revenue += expected - refund
```

```
    order_results.append((
        order["id"], product, subtotal, discount, expected,
        order["paid"], difference, dispatch_qty, return_qty, refund
    ))

print("INTEGRATED E-COMMERCE ORDER REPORT")
print("-" * 86)
for result in order_results:
    if len(result) == 2:
        print(result[0], result[1])
    else:
        oid, product, subtotal, discount, expected, paid, diff, dispatched, returned, refund = result
        status = "MATCH" if abs(diff) <= 0.01 else "MISMATCH"
        print(f"{oid} | {product} | subtotal Rs.{subtotal:.2f} | "
              f"discount Rs.{discount:.2f} | expected Rs.{expected:.2f} | "
              f"paid Rs.{paid:.2f} | {status} ({diff:+.2f}) | "
              f"dispatched {dispatched} | returned {returned} | "
              f"refund Rs.{refund:.2f}")

print("-" * 86)
print(f"Gross sales before discount : Rs.{total_gross:.2f}")
print(f"Total discounts             : Rs.{total_discounts:.2f}")
print(f"Total refunds               : Rs.{total_refunds:.2f}")
print(f"Net revenue after refunds   : Rs.{net_revenue:.2f}")
print(f"Payment discrepancies       : {len(discrepancies)}")
for oid, expected, paid, diff in discrepancies:
    print(f"  {oid}: expected Rs.{expected:.2f}, paid Rs.{paid:.2f}, difference Rs.{diff:+.2f}")

print("\nFINAL INVENTORY")
for product, details in inventory.items():
    print(f"{product:12s} | unit price Rs.{details['price']:.2f} | "
          f"closing stock {details['stock']} units")
```

8. Test Cases

Test	Input / Condition	Expected Result	Observed Result
TC-01	2 keyboards at ■800, 10% discount	Subtotal ■1,600; discount ■160; expected payment ■1,440	Pass
TC-02	Payment ■1,440 for ORD101	Payment matches expected amount	Pass
TC-03	Headphones: price ■1,500, paid ■1,400	Flag discrepancy of -■100	Pass
TC-04	Return 1 of 2 discounted keyboards	Refund ■720; returned unit added to stock	Pass
TC-05	Dispatch quantity greater than available stock	Report insufficient stock; do not dispatch	Covered by validation logic
TC-06	Return quantity greater than dispatched quantity	Report invalid return; roll back stock deduction	Covered by validation logic

9. Sample Output Screenshots

The following console-style screenshots are illustrative renderings of the program output for the sample data above. They are included in the report as output evidence; run the Python code to capture a screenshot from your own Python environment.

```
INTEGRATED E-COMMERCE ORDER REPORT
--------------------------------------------------------------------------------
ORD101 | Keyboard | subtotal Rs.1600.00 | discount Rs.160.00 | expected Rs.1440.00 | paid Rs.1440.00 | MATCH (+0.00) | dispatched 2 | retu
ORD102 | Mouse | subtotal Rs.1500.00 | discount Rs.75.00 | expected Rs.1425.00 | paid Rs.1425.00 | MATCH (+0.00) | dispatched 3 | returned
ORD103 | Headphones | subtotal Rs.1500.00 | discount Rs.0.00 | expected Rs.1500.00 | paid Rs.1400.00 | MISMATCH (-100.00) | dispatched 1 |
ORD104 | Keyboard | subtotal Rs.800.00 | discount Rs.0.00 | expected Rs.800.00 | paid Rs.800.00 | MATCH (+0.00) | dispatched 1 | returned
--------------------------------------------------------------------------------
Gross sales before discount : Rs.5400.00
Total discounts             : Rs.235.00
Total refunds               : Rs.720.00
Net revenue after refunds   : Rs.2945.00
Payment discrepancies       : 1
  ORD103: expected Rs.1500.00, paid Rs.1400.00, difference Rs.-100.00

FINAL INVENTORY
Keyboard     | unit price Rs.800.00 | closing stock 18 units
Mouse        | unit price Rs.500.00 | closing stock 12 units
Headphones   | unit price Rs.1500.00 | closing stock 9 units
```

Note: The net revenue shown is the sum of discounted expected order totals less refunds. The ■100 payment discrepancy is reported separately; actual cash received is lower than expected by ■100.

10. Results and Discussion

The sample run processes four orders. Three payments match their expected totals, while ORD103 has a ■100 short payment. A 10% discount on two keyboards produces a ■160 discount, and the return of one keyboard produces a ■720 refund. Closing stock is 18 keyboards, 12 mice, and 9 headphones. The report separates order reconciliation from net revenue calculation so that a payment mismatch remains visible rather than silently changing the order value.

11. Conclusion

The Integrated E-Commerce Order Processing and Reconciliation program demonstrates how Python dictionaries, loops, conditions, arithmetic operations, and formatted output can automate basic order management. It applies discounts, tracks dispatches and returns, updates stock, calculates revenue after refunds, and highlights payment discrepancies. For a real production system, the program should be extended with a database, unique transaction IDs, tax and shipping calculations, role-based access, audit logs, and integration with payment and courier APIs.

12. How to Run the Program

- Install Python 3 on a computer or use an online Python editor.
- Copy the source code from Section 7 into a file named ecommerce_reconciliation.py.
- Run: python ecommerce_reconciliation.py
- Compare the console output with the sample output in Section 9.